Smart Chef & Pantry AI

Database Design & ER Model

Sürüm 1.0

---

1. Veritabanı Teknolojisi

Ana veritabanı:

PostgreSQL

Backend ORM:

SQLAlchemy

Migration:

Flask-Migrate / Alembic

Veritabanı katmanı Flask uygulamasından doğrudan UI tarafından erişilebilir olmayacaktır.

Akış:

Flutter
↓
REST API
↓
Flask Service Layer
↓
Repository / SQLAlchemy
↓
PostgreSQL

---

2. Temel Veri Modeli

MVP'de aşağıdaki temel tablolar kullanılacaktır:

users

ingredients
recipes
recipe_ingredients
favorites
cart_items

İlişkiler:

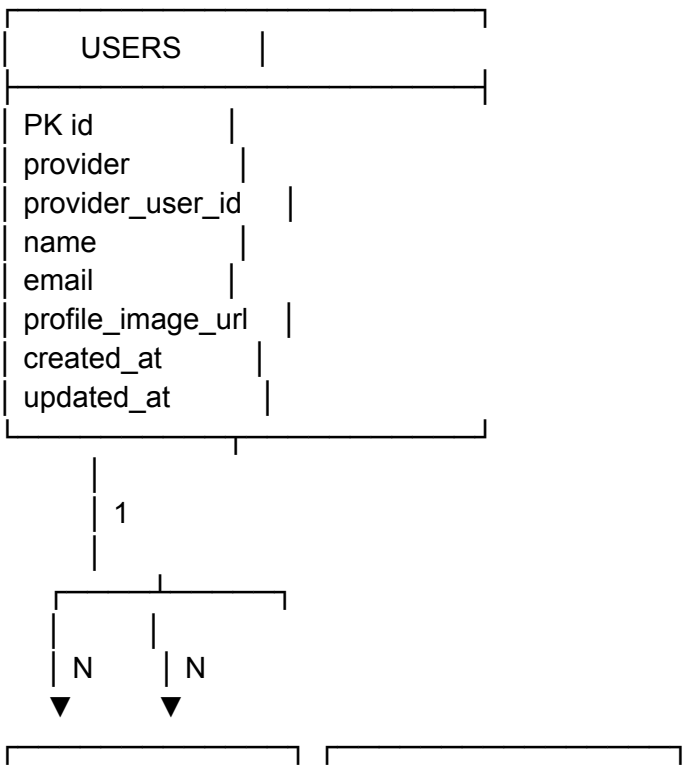
User
 │
 ├──────────────────────< Favorite >──────────────────── Recipe
 │
 └──────────────────────< CartItem >──────────────────── Ingredient

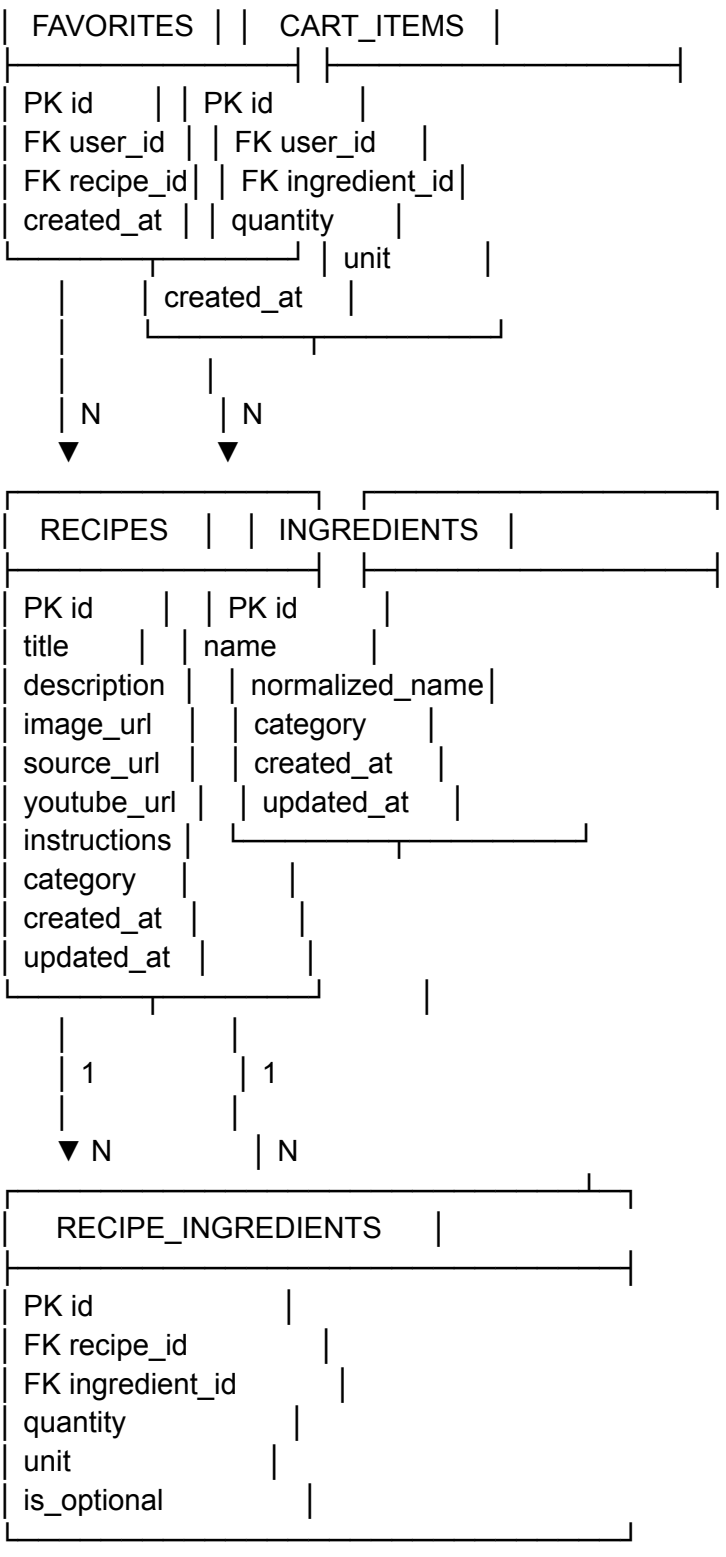
Recipe
 │
 └──────────────────────< RecipeIngredient >──────── Ingredient

Böylece Recipe ve Ingredient arasında **many-to-many** ilişki kurulmuş olur.

---

3. ER Diagram





---

4. USERS

Kullanıcının kimlik ve profil bilgilerinin tutulduğu tablodur.

Alanlar

Alan	Tip	Null	Açıklama
id	UUID	Hayır	Primary Key
provider	VARCHAR	Hayır	Google vb.
provider_user_id	VARCHAR	Hayır	Sosyal sağlayıcıdaki kullanıcı ID
name	VARCHAR	Hayır	Kullanıcı adı
email	VARCHAR	Hayır	E-posta
profile_image_url	TEXT	Evet	Profil fotoğrafı
created_at	TIMESTAMP	Hayır	Oluşturulma tarihi
updated_at	TIMESTAMP	Hayır	Güncellenme tarihi

Kurallar

`provider + provider_user_id` benzersiz olmalıdır.

Email için de uygun bir unique constraint kullanılmalıdır.

---

5. INGREDIENTS

Sistemin standart malzeme sözlüğüdür.

Örneğin:

Domates
Yumurta
Soğan
Biber
Tavuk
Patates
Peynir

Alanlar

Alan	Tip	Null	Açıklama
id	UUID	Hayır	Primary Key

name	VARCHAR	Hayır	Görünen isim
normalized_name	VARCHAR	Hayır	Eşleme için normalize edilmiş isim
category	VARCHAR	Evet	Sebze, et, süt ürünü vb.
created_at	TIMESTAMP	Hayır	Oluşturulma
updated_at	TIMESTAMP	Hayır	Güncellenme

Neden `name` ve `normalized_name` ayrı?

Örneğin:

name:
Domates

normalized_name:
domates

veya gelecekte:

name:
Çeri Domates

normalized_name:
domates

gibi eşleme mantıkları kurulabilir.

Ancak burada dikkat:

`normalized_name` alanı tek başına semantik eş anlamlılık çözmez.

Örneğin:

domates
çeri domates
domates sosu

otomatik olarak aynı malzeme kabul edilmemelidir.

Bu ayrım AI/matching katmanında ayrıca yönetilecektir.

---

6. RECIPES

Tariflerin ana tablosudur.

Alanlar

Alan	Tip	Null	Açıklama
id	UUID	Hayır	Primary Key
title	VARCHAR	Hayır	Tarif adı
description	TEXT	Evet	Açıklama
image_url	TEXT	Evet	Tarif görseli
instructions	TEXT	Hayır	Hazırlama adımları
source_url	TEXT	Evet	Orijinal tarif
youtube_url	TEXT	Evet	YouTube videosu
category	VARCHAR	Evet	Ana kategori
created_at	TIMESTAMP	Hayır	Oluşturulma
updated_at	TIMESTAMP	Hayır	Güncellenme

---

7. RECIPE_INGREDIENTS

Bu tablo sistemin en önemli tablolarından biridir.

Çünkü bir tarif birden fazla malzeme içerir ve bir malzeme birçok tarifte kullanılabilir.

Dolayısıyla:

Recipe ↔ Ingredient

ilişkisi:

Many-to-Many

olmalıdır.

Bunu `recipe_ingredients` association table çözer.

Alanlar

Alan	Tip	Null	Açıklama
id	UUID	Hayır	Primary Key
recipe_id	UUID	Hayır	Recipe FK
ingredient_id	UUID	Hayır	Ingredient FK
quantity	DECIMAL	Evet	Miktar
unit	VARCHAR	Evet	adet, gram, ml vb.
is_optional	BOOLEAN	Hayır	Opsiyonel mi?

Örnek

Menemen:

recipe_id = 101

ingredient_id	quantity	unit
domates	3	adet
yumurta	3	adet
soğan	1	adet
biber	2	adet

---

8. FAVORITES

Kullanıcının favoriye aldığı tarifleri tutar.

Alanlar

Alan	Tip	Null
id	UUID	Hayır
user_id	UUID	Hayır
recipe_id	UUID	Hayır
created_at	TIMESTAMP	Hayır

Kritik constraint

Aynı kullanıcı aynı tarifi iki kez favorileyememelidir.

Bu nedenle:

UNIQUE(user_id, recipe_id)

kullanılmalıdır.

---

9. CART_ITEMS

Kullanıcının alışveriş listesindeki malzemeleri tutar.

Alanlar

Alan	Tip	Null
id	UUID	Hayır
user_id	UUID	Hayır
ingredient_id	UUID	Hayır
quantity	DECIMAL	Hayır
unit	VARCHAR	Evet
created_at	TIMESTAMP	Hayır
updated_at	TIMESTAMP	Hayır

---

10. Sepet Birleştirme Kuralı

Kritik iş kuralı:

UNIQUE(user_id, ingredient_id, unit)

kullanılabilir.

Örneğin:

User 1
Domates
2 adet

daha sonra:

Domates
3 adet

eklerse:

Domates
5 adet

olur.

Yeni satır oluşturulmaz.

---

11. Quantity ve Unit Tasarımı

Burada ileride ciddi problem yaşamamak için miktarı string olarak saklamıyoruz.

Kötü:

quantity = "2-3 adet"

Daha iyi:

quantity = 2
unit = "adet"

Ancak bazı tariflerde:

"bir tutam tuz"
"göz kararı"
"yarım limon"

gibi miktarlar bulunabilir.

Bu nedenle MVP'de iki yaklaşım değerlendirilebilir.

Yaklaşım A — Structured

quantity = 0.5

unit = "adet"

Yaklaşım B — Structured + Display

quantity = 0.5
unit = "adet"
display_quantity = "yarım"

İkinci yaklaşım daha esnektir.

Bu nedenle `recipe_ingredients` tablosunda:

quantity
unit
display_quantity

alanlarının kullanılması önerilir.

---

12. Recipe Matching İçin Veri Akışı

Kullanıcı:

Domates
Yumurta
Soğan

seçtiğinde:

Flutter
↓
ingredient IDs
↓
Flask
↓
Recipe Matching Service
↓
recipe_ingredients
↓
Matching Score

Örneğin:

Tarif A
4 gerekli malzeme
3 eşleşme

Score = 3 / 4
= %75

Eksik malzeme:

Biber

olarak belirlenir.

---

13. Optional Ingredient

`is_optional` alanı önemlidir.

Örneğin:

Menemen

Domates required
Yumurta required
Soğan optional
Biber optional

Kullanıcıda soğan yoksa tarif tamamen başarısız sayılmamalıdır.

Bu nedenle matching algoritması:

Required ingredients
+
Optional ingredients

ayrımını dikkate almalıdır.

---

14. Kategori Yapısı

MVP'de kategori için ayrı tablo açmak zorunda değiliz.

Başlangıçta:

recipes.category

yeterlidir.

Örnek:

Kahvaltı
Ana Yemek
Çorba
Salata
Tatlı
Atıştırmalık

Proje büyürse:

categories

tablosuna ayrılabilir.

---

15. Favorite ve Cart Neden User'a Bağlı?

Kullanıcının:

Favorites
Cart

verileri cihazda değil backend'de tutulur.

Örneğin:

User A
 ├── Favorite: Menemen
 └── Cart: Domates

User B
 ├── Favorite: Pizza
 └── Cart: Peynir

Bir kullanıcının verisi diğer kullanıcıya erişilemez.

---

16. Delete Behavior

Tarif veya malzeme silindiğinde ilişkili kayıtların davranışı belirlenmelidir.

Önerilen:

Recipe silinirse

İlgili:

recipe_ingredients
favorites

kayıtları temizlenmelidir.

Ingredient silinirse

İlgili:

recipe_ingredients
cart_items

kayıtları temizlenmelidir.

Ancak gerçek uygulamada tarifleri fiziksel olarak silmek yerine admin tarafında:

is_active

gibi bir alan kullanmak daha güvenli olabilir.

Bu nedenle tarif ve malzemelerde ileride soft-delete / active-state desteği eklenebilir.

---

17. Indexler

Performans için aşağıdaki alanlar indexlenmelidir:

ingredients.normalized_name

recipes.category

recipe_ingredients.recipe_id

recipe_ingredients.ingredient_id

favorites.user_id

favorites.recipe_id

cart_items.user_id

cart_items.ingredient_id

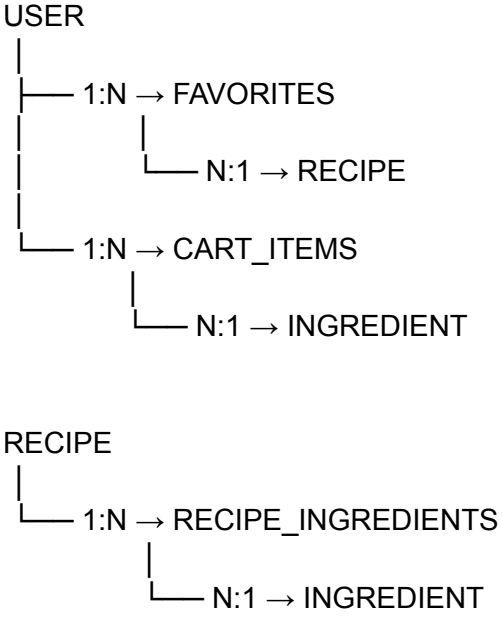
Özellikle ingredient matching sırasında:

recipe_ingredients.ingredient_id

çok sık sorgulanacaktır.

---

18. Veritabanı İlişki Özeti



---

19. Nihai MVP Tablo Seti

users
ingredients
recipes
recipe_ingredients
favorites
cart_items

MVP için bundan fazlasını eklemiyoruz.

Özellikle:

orders
payments
reviews
comments
recommendation_history
user_preferences

gibi tabloları şu anda açmıyoruz.

Bunlar ileride gerçekten ihtiyaç oluştuğunda eklenecek.

---

20. Veri Modelinin Projedeki Rolü

Veritabanı sadece CRUD yapmak için kullanılmayacaktır.

Özellikle:

recipe_ingredients
↓
Ingredient Matching
↓
Recipe Score
↓
Recommendation

sistemin AI/recommendation kısmının temel veri kaynağı olacaktır.

Bu nedenle `recipe_ingredients` tablosu projenin çekirdek veri modelidir.